Chapter 08: Pointers

Khiem Nguyen – khiem.nguyen@glasgow.ac.uk

Contents

1 Introduction to pointers **1**
1.1 Memory and address . . . 1
1.2 The address-of operator (&): The ampersand character . . . 1
1.3 The dereference operator (*): The asterisk character . . . 2
1.4 Address-of and dereference operators: Some points to note . . . 2

2 Pointers **3**
2.1 Pointers . . . 3
2.2 Pointers: Example . . . 3
2.3 Pointers: Declare multiple pointers in one line . . . 4
2.4 Pointers: Demonstration . . . 4

3 Pointer initialization **4**
3.1 Pointer initialization . . . 4
3.2 Pointer initialization . . . 5
3.3 Pointer initialization: Demonstration . . . 5

4 Pointers and Assignment **5**
4.1 Pointers and assignment . . . 5
4.2 Pointers and assignment . . . 6
4.3 Pointers and assignment: Key insight . . . 6
4.4 Pointers behave much like lvalue reference . . . 7
4.5 Pointers behave much like lvalue reference: Figure demonstration . . . 8
4.6 But references are not pointers . . . 8

5 The address-of operator returns a pointer **8**
5.1 The address-of operator returns a pointer . . . 8
5.2 The size of pointers . . . 9
5.3 The size of pointers . . . 9

1 Introduction to pointers

1.1 Memory and address

Consider a normal variable like this:

```
int x { 425 };
```

Behind the scene

- When this code is executed, a piece of memory from RAM will be assigned to this object.

- The variable `x` is (hypothetically, see the figure before) assigned memory address `0x1234`.
- Whenever we use the variable `x`, the program will go to the memory `0x1234` to access the value stored there.

This is also true with references:

```
int main()
{
    int x { 42 };
    int& ref { x };
    std::cout << "x = " << x << ", ref = " << ref << '\n';
    return 0;
}
```

Whenever we use `ref`, the program will go to the memory address of the variable `x` (hypothetically address = `0x1234`) to access the value.

1.2 The address-of operator (&): The ampersand character

The **address-of operator (&)** returns the memory of address of its operand.

Listing 1 We can retrieve the address of a variable by using the address-of operator, the ampersand character &. Filename=`address_of_operator_example.cpp`

```
#include <iostream>
int main()
{
    int ivalue{ 5 };
    std::cout << ivalue << '\n';  // print the value of variable ivalue
    std::cout << &ivalue << '\n'; // print the memory address of variable ivalue

    double dvalue { 42 };
    std::cout << &dvalue << '\n';
    return 0;
}
```

Let us run this code snippet.

- Memory addresses are typically printed as hexadecimal values.
- For objects that use more than one byte of memory, address-of will return the memory address of the first byte used by the object.

1.3 The dereference operator (*): The asterisk character

The **dereference operator (*)** (also occasionally called the **indirection operator**) returns the value at a given memory address as an lvalue.

Let us run this code snippet.

Listing 2 Dereference operator returns the value at a given memory address. Filename=`dereference_operator_example.cpp`

```cpp
#include <iostream>

int main()
{
    double x{ 5 };
    std::cout << x << '\n';  // print the value of variable x
    std::cout << &x << '\n'; // print the memory address of variable x

    // print the value at the memory address of variable x
    // (parentheses not required, but make it easier to read)
    std::cout << *(&x) << '\n';

    return 0;
}
```

1.4 Address-of and dereference operators: Some points to note

Key insight

The *address-of operator* (&) and *dereference operator* (*) work as opposites: address-of gets the address of an object, and dereference gets the object at an address.

Regarding the ampersand

The `&` may cause confusion because it has different meanings depending on context:

- When following a type name, `&` denotes an lvalue reference:

  `int& ref`.

- When used in a unary context in an expression, `&` is the address-of operator:

  `std::cout << &x`.

- When used in a binary context in an expression, `&` is the Bitwise AND operator:

  `std::cout << x & y`.

2 Pointers

A pointer is an object that holds a memory address (typically of another variable) as its value.

2.1 Pointers

A **pointer** is an object that holds a memory address (typically of another variable) as its value.

A type that specifies a pointer is called a **pointer type**.

Pointer types are declared using an asterisk (*).

```
double;     // a normal double
double&;    // an lvalue reference to a double value
double*;    // a pointer to an double value (holds the address of a double value)
```

2.2 Pointers: Example

Listing 3 An example of using pointers, address-of operator and dereference operator. Filename=`pointer_example.cpp`

```
#include <iostream>

int main()
{
    double x { 42 };    // normal variable
    double& ref { x }; // a reference to a double (bound to x)

    double* ptr1;       // a pointer to a double
    ptr1 = &x;
    std::cout << "ptr1 = " << ptr1 << '\n';   // show some address
    std::cout << "value stored at ptr1 = " << *ptr1 << '\n';

    double* ptr2 = &ref;
    std::cout << "ptr2 = " << ptr2 << '\n';
    std::cout << "value stored at ptr2 = " << *ptr2 << '\n';

    return 0;
}
```

Let us run this code snippet.

Question: What would you expect to see?

Note: The asterisk is part of the declaration syntax for pointers, not the use of dereference.

2.3 Pointers: Declare multiple pointers in one line

We generally should not declare multiple variables on a single line.

If we do, the asterisk has to be included with each variable.

```
int* ptr1, ptr2;    // incorrect: ptr1 is a pointer to an int,
                    // but ptr2 is just a plain int!
int* ptr3, * ptr4;  // correct: ptr3 and ptr4 are both pointers to an int
```

2.4 Pointers: Demonstration

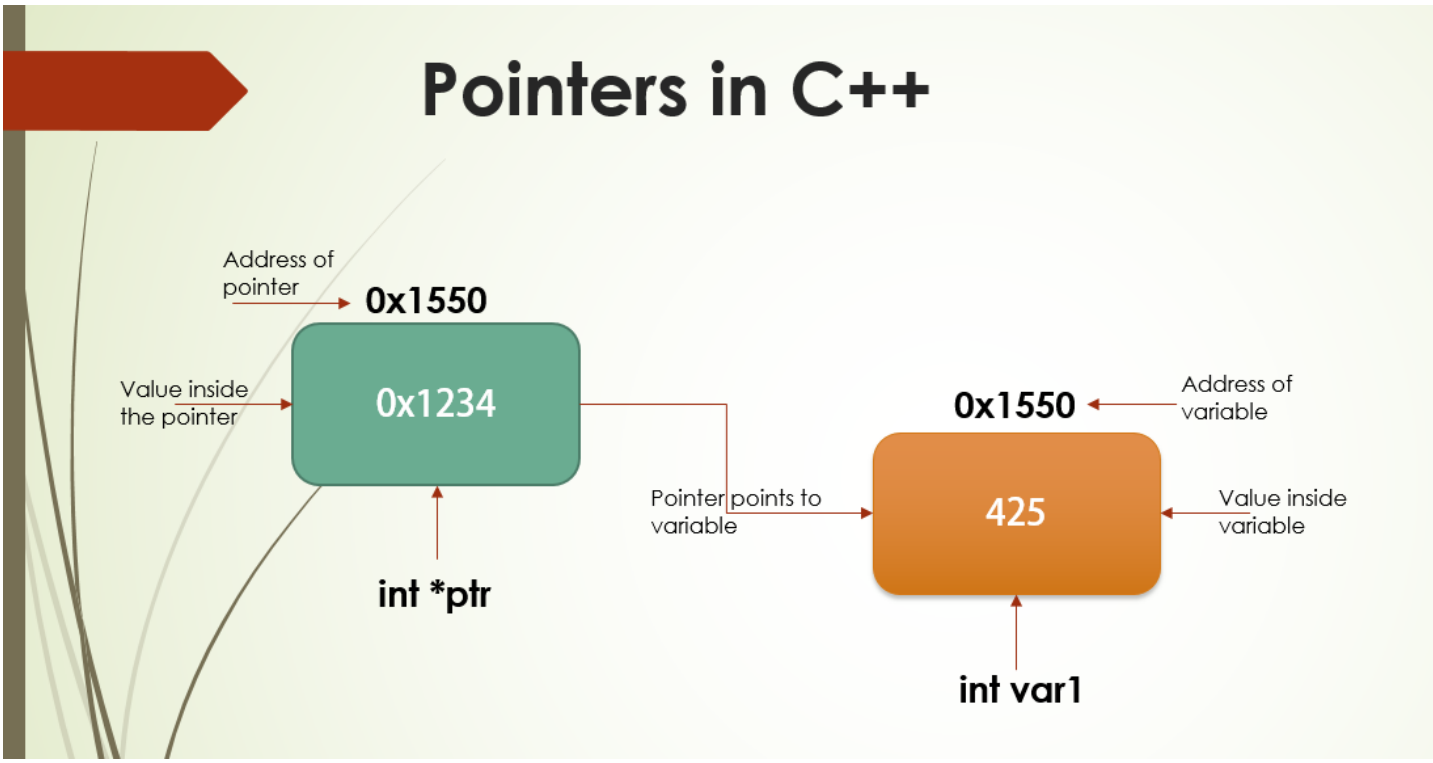


Figure 1: This is an excellent demonstration of a pointer in C++

3 Pointer initialization

A pointer that has not been initialized is sometimes called a **wild pointer**.

Always initialize your pointers.

3.1 Pointer initialization

- A pointer that has not been initialized is sometimes called a **wild pointer**.
- Wild pointers contain garbage address.
- Dereferencing a wild pointer will result in undefined behavior (pretty much like using uninitialized variables)

Best practice

Always initialize your pointers.

3.2 Pointer initialization

Since pointers hold addresses, when we initialize or assign a value to a pointer, that value has to be an address – Just common sense.

Listing 4 Best practice: Always initialize pointers. Filename=`initialize_pointers.cpp`

```cpp
int main()
{
    int x{ 5 };

    int* ptr;        // an uninitialized pointer (holds a garbage address)
    int* ptr2{};     // a null pointer (we'll discuss these in the next lesson)
    int* ptr3{ &x }; // a pointer initialized with the address of variable x

    return 0;
}
```

Listing 5 Initialize pointers using addresses. Filename=`initialize_pointers_using_addresses.cpp`

```cpp
#include <iostream>

int main()
{
    int x{ 5 };
    std::cout << x << '\n';     // print the value of variable x

    int* ptr{ &x };             // ptr holds the address of x
    std::cout << *ptr << '\n';  // use dereference operator to print the
                                // value at the address that ptr is holding
                                // (which is x's address)

    return 0;
}
```

3.3 Pointer initialization: Demonstration

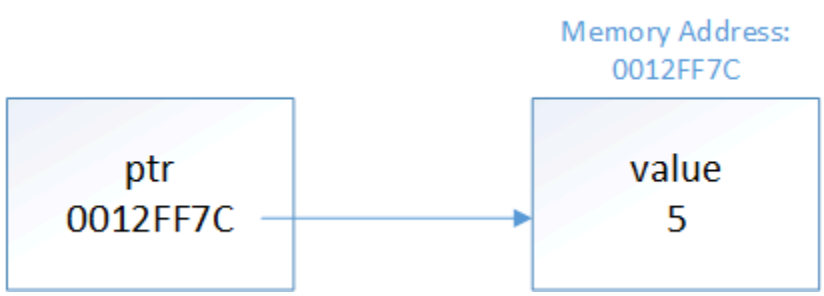


Figure 2: Repeat the pointer concept in figure

This is where **pointers** get their name from:

- `ptr` is holding address of `x`.
- `ptr` is **pointing to** `x`.

4 Pointers and Assignment

Because pointers are objects, we can perform assignment on them.

4.1 Pointers and assignment

Use assignment with pointers in two ways:

- To change what the pointer is pointing at: Assign the pointer a new address.
- To change the value being pointed at: Assign the *dereferenced* pointer a new value

Listing 6 Assign a pointer to a new address. Filename=`assign_pointer_to_new_address.cpp`

```cpp
#include <iostream>
int main()
{
    double x { 42.0 };
    double* ptr{ &x }; // ptr initialized to point at x
    std::cout << *ptr << '\n'; // print the value at the address being pointed to (x's address)
    *ptr = 10;
    std::cout << x << '\n';
    double y{ 24.0 };
    ptr = &y; // // change ptr to point at y
    std::cout << *ptr << '\n'; // print the value at the address being pointed to (y's address)
    *ptr = 12;

    std::cout << y << '\n';
    return 0;
}
```

Let us run this code snippet. **Question** Write code to change the value of `x` to `24` via using `ptr`.

4.2 Pointers and assignment

4.3 Pointers and assignment: Key insight

Key insight

- When we use a pointer without a dereference (`ptr`), we are accessing the address held by the pointer. Modifying this (`ptr = &y;`) changes what the pointer is pointing at.
- When we dereference a pointer (`*ptr`), we are accessing the object being pointed at. Modifying this (`*ptr = 6;`) changes the value of the object being pointed at.

At this point, you should already feel:

Pointers behave much like lvalue references

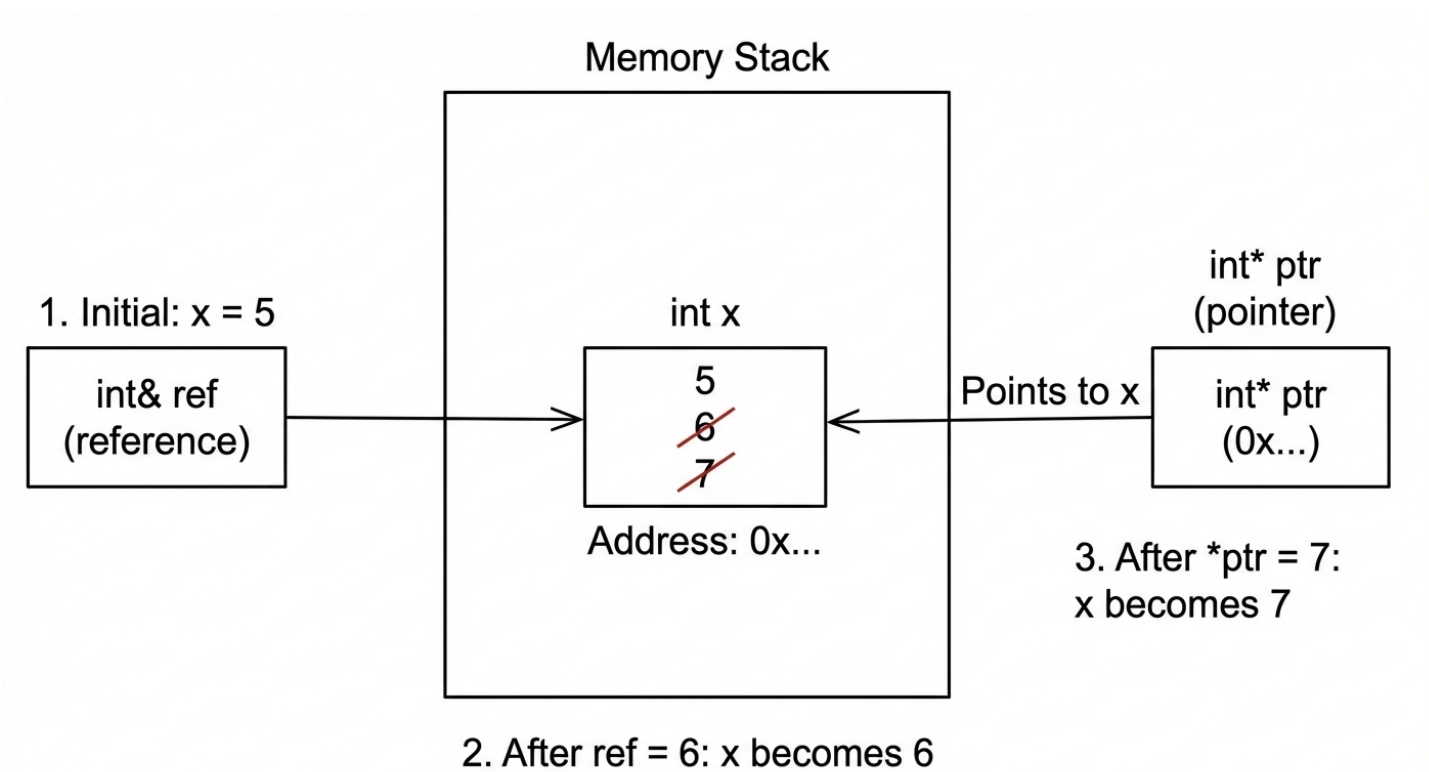
4.4 Pointers behave much like lvalue reference

Important to note: pointer is not lvalue reference. But we can write code like this:

Listing 7 Change value of variable via dereferencing a pointer. Filename=`change_value_using_dereference.cpp`

```cpp
#include <iostream>

int main()
{
    double x{ 42 };
    double* ptr{ &x };          // initialize ptr with address of variable x

    std::cout << x << '\n';     // print x's value
    std::cout << *ptr << '\n';  // print the value at the address that ptr is
                                // holding (x's address)

    *ptr = 24;  // The object at the address held by ptr (x)
                // assigned value 24 (note that ptr is dereferenced here)

    std::cout << x << '\n';
    std::cout << *ptr << '\n';  // print the value at the address that ptr is
                                // holding (x's address)


    return 0;
}
```

4.5 Pointers behave much like lvalue reference: Figure demonstration



4.6 But references are not pointers

There are some other differences between pointers:

References	Pointers
must be initialized	not required to be initialized
are not objects	are objects
cannot be reseated	can change what they are pointing at
must always be bound to an object	can point to nothing (null pointer)
"safe" (outside of dangling reference)	are inherently dangerous (we'll discuss in the next lesson)

5 The address-of operator returns a pointer

5.1 The address-of operator returns a pointer

- The address-of operator *does not* return the address of its operand *as a literal*.
- It returns a pointer to the operand (whose value is the address of the operand).

Example: Given variable `int x`, `&x` returns an `int*` holding the address of `x`.

Let us run this code snippet. **Question**: What do you think about the size of pointers?

5.2 The size of pointers

The size of a pointer is dependent upon the architecture the executable is compiled for:

- a pointer on a 32-bit machine is 32 bits (4 bytes)
- a pointer on a 64-bit machine is 64 bits (8 bytes)

This is true regardless of the size of the object being pointed to.

This is just common sense: A pointer is just a memory address, and the number of bits needed to access a memory address is constant.

5.3 The size of pointers

Let us run this code snippet.

Listing 8 Pointers behave much like lvalue references. But they are not the same. Filename=`pointers_behave_like_references.cpp`

```cpp
#include <iostream>

int main()
{
    int x{ 5 };
    int& ref { x };  // get a reference to x
    int* ptr { &x }; // get a pointer to x

    std::cout << x;
    std::cout << ref;  // use the reference to print x's value (5)
    std::cout << *ptr << '\n'; // use the pointer to print x's value (5)

    ref = 6; // use the reference to change the value of x
    std::cout << x;
    std::cout << ref;  // use the reference to print x's value (6)
    std::cout << *ptr << '\n'; // use the pointer to print x's value (6)

    *ptr = 7; // use the pointer to change the value of x
    std::cout << x;
    std::cout << ref;  // use the reference to print x's value (7)
    std::cout << *ptr << '\n'; // use the pointer to print x's value (7)

    return 0;
}
```

Listing 9 The address-of operator does not return the address of its operand as a literal. It returns a pointer to the operand. Filename=`address_of_returns_pointer.cpp`

```cpp
#include <iostream>
#include <typeinfo>

int main()
{
    int* x { nullptr };
    std::cout << typeid(x).name() << '\n';  // print the type of x
    std::cout << typeid(&x).name() << '\n'; // print the type of &x

    return 0;
}
```

Listing 10 Size of pointers depends on the architecture the executable is compiled for. Filename=`size_of_pointers.cpp`

```cpp
#include <iostream>

int main() // assume a 32-bit application
{
    char* char_ptr{};        // chars are 1 byte
    int* int_ptr{};          // ints are usually 4 bytes
    long double* ld_ptr{};   // long doubles are usually 8 or 12 bytes

    std::cout << sizeof(char_ptr) << '\n';  // prints 8 on my computer
    std::cout << sizeof(int_ptr) << '\n';   // prints 8
    std::cout << sizeof(ld_ptr) << '\n';    // prints 8

    return 0;
}
```